



Bob S22037 3 Jul 2016

Integrando impressora bluetooth portátil com aplicativos Delphi Multi-Device Android

Neste artigo mostraremos como utilizar uma mini impressora térmica bluetooth portátil em um aplicativo Android desenvolvido em Delphi através da integração com o SDK da impressora.

Inicialmente devemos ter em mente que cada impressora possui uma forma própria de implementação, algumas mais parecidas entre si que outras, mas divergindo de fabricante para fabricante. E como cada um possui um SDK específico para sua impressora, os métodos e os parâmetros para instância dos objetos variam entre os diversos fabricantes.

Quem já desenvolveu algum sistema de PDV sabe bem como é ter que lidar com vários fabricantes de impressoras fiscais, as impressoras bluetooth seguem a mesma logica.

Neste artigo faremos a integração com um modelo “Chig-Ling”, a impressora da Ocom modelo OCPP-M03, (<http://www.ocominc.com/pt/products/OCPP-M03.htm>) que custa em torno de U\$ 150,00 com o envio pelo Aliexpress, sem impostos!

A primeira coisa que precisamos fazer é obter o SDK da impressora para o Android.

Enquanto que para o Windows os fabricantes nos disponibilizam arquivos .dll para integração de seus dispositivos, no Android os fabricantes nos disponibilizam arquivos .jar.

Jar - (Java ARchive) é um arquivo compactado usado para distribuir um conjunto de classes Java, um aplicativo java, ou outros itens como imagens, XMLs, entre outros. É usado para armazenar

Sim, é Java!

E se você souber pelo menos ler código Java, ajudará bastante a entender o funcionamento do SDK da impressora. Uma vez que os exemplo de integração com Android serão sempre disponibilizados em Java.

Arquivo Bridge

Uma vez com o SDK em mãos, no caso do modelo OCPP-M03 o fabricante nos disponibilizou o arquivo `btsdk.jar`, vamos precisar consumir este arquivo em nosso projeto Firemonkey no Delphi.

Para utilizarmos um arquivo jar precisamos gerar uma Unit que faça o meio de campo entre nosso código fonte e o arquivo java, essa unit é chamada de "Delphi native bridge file", ou simplesmente um arquivo de Ponte ou Bridge.

Desde a versão XE7 a Embarcadero disponibiliza uma ferramenta para ajudar nós desenvolvedores Delphi na criação de interfaces com as classes Java Android.

O Java2Op tem a finalidade de traduzir as APIs Java para Object Pascal, ou seja criar nosso arquivo `Bridge`.

Mas antes de utilizarmos o Java2Op é preciso alguns pré requisitos, na sequencia veremos como baixá-lo e executá-lo.

Java JDK

Para executar o Java2Op é necessário a instalação da JDK do Java. Mas atenha-se a versão, o Java2Op utiliza a versão 7 da JDK e não a 8, atual versão distribuída pela Oracle.

Você pode baixar a versão 7 do JDK do Java no seguinte link:

<http://www.oracle.com/technetwork/java/javase/downloads/java-archive-downloads-javase7-521261.html>

Requisito para baixar versões antigas, para efetuar o download da JDK 7 é preciso se cadastrar-se no site da Oracle.

Após a instalação da JDK é preciso configurar o Path do Windows (Painel de Controle/Sistema e Segurança/Sistema/Configurações avançadas do sistema/Variáveis de Ambiente) adicionando a pasta `bin` do diretório de instalação.

Java2Op

O Java2Op é disponibilizado pela Embarcadero para usuários registrados:

<http://cc.embarcadero.com/Download.aspx?id=30007>

Ao baixarmos o Java2Op, um arquivo zip e descarregado, neste arquivo vamos precisar apenas do java2op.zip. Descompacte-o em um diretório seu.

Para fazer a geração do Bridge copie o arquivo jar para uma pasta src no diretório do Java2op. Como ele é uma ferramenta de linha de comando, então faremos sua execução via prompt de comando. CMD neles!

O seguinte comando efetua a geração do Bridge:

```
Java2op.exe -jar <ArquivoJava.jar> -unit <ArquivoBridge.pas>
```

Como no meu caso o arquivo .jar se chama btsdk.jar, está na pasta src no diretório do Java2Op e vou chamar o meu arquivo Bridge de Android.JNI.btsdk.pas, seguindo a convenção de nomenclatura de Units Android no Delphi, o comando fica da seguinte forma:

```
Java2op.exe -jar srcbtsdk.jar -unit Android.JNI.btsdk.pas
```

Caso ao executá-lo lhe aparecer o este erro: **"Generic Type not defined"**

Você provavelmente está utilizando a versão 8 da JDK e não a 7. Verifique a instalação e se o seu Path do Windows estão apontando para versão correta.

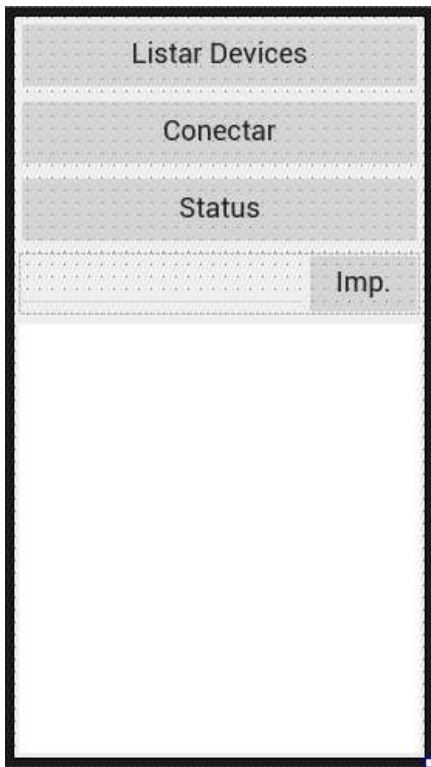
Tudo ocorrendo bem o arquivo Android.JNI.btsdk.pas foi criado na pasta do Java2Op.

Tanto o btsdk.jar quanto o Android.JNI.btsdk.pas serão utilizados em nosso projeto.

Desenvolvendo o aplicativo

Vamos ao nosso projeto!

Criaremos então nosso projeto Firemonkey (FileNewMulti-Device ApplicationBank Application) e nele colocaremos quatro botoes, um edit e um memo.

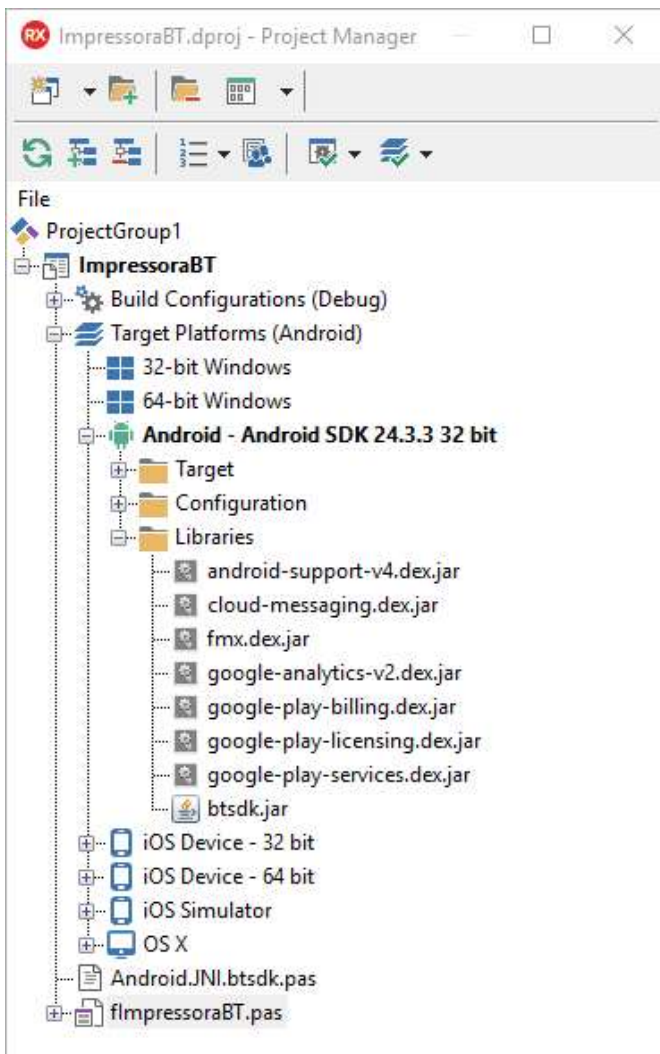


E adicionaremos nossos dois arquivos ao projeto.

O Arquivo `Android.JNI.btsdk.pas` é adicionado normalmente como qualquer outro fonte, ou seja com botão direito do mouse sobre o nome do projeto no Project Manager, clicamos na opção `Add...`, e selecionamos o nosso `.pas`.

O arquivo `btsdk.jar` será adicionado às bibliotecas do Android no Projeto.

No Project Manager, na pasta `Target Platform/Android/Libraries`, com o botão direito clicamos na opção `Add...` e selecione o arquivo `.jar`



Com estes passos as bibliotecas necessárias para a integração da impressora foram efetuados. Iremos para a parte da codificação.

Integrando o SDK

Na maior parte dos casos os fabricante disponibilizam um código de exemplo sobre como fazer a integração com seus dispositivos. Estes exemplos, assim como a biblioteca estarão desenvolvidos em Java. Como disse antes, saber pelo menos ler códigos fontes em Java será muito útil.

A biblioteca que importamos expõem uma interface chamada JBluetoothService, ela quem faz toda a “mágica” para nós, dela iremos utilizar 4 funcionalidade em nosso projeto, que será:

- 1 - Listar os dispositivos pareados;
- 2 - Conexão com a Impressora;
- 3 - Status da Impressora;
- 4 - Envio de texto para impressão.

Dessa forma começamos declarando uma variável Global Privada do tipo JBluetoothService, iremos chama-la de FService.

```
[...]  
private  
    FService: JBluetoothService;
```

A instância de nosso objeto principal se dará no clique do botão que listará os Devices pareados do mobile (btnDevices).

JavaClass.Init()

Quando trabalhamos com arquivos Bridge, onde nossas classe são de origem java, temos que esquecer um pouco a forma que estamos acostumado a instanciar nossos objetos. Ao invés de utilizarmos o Construtor padrão, o método Create, iremos utilizar o método JavaClass.Init().

A Instância da variavel FService, será feita através da classe TJBluetoothService, neste caso TJBluetoothService.JavaClass.init(...).

```
//  
function init(P1: JContext; P2: JHandler): JBluetoothService; cdecl;
```

O Método TJBluetoothService.JavaClass.Init exige dois parâmetros, "P1: JContext" e "P2: JHandler".

Grande parte dos dispositivos que são integrados no Android apenas exigem apenas que seja informado a Activity responsável pelo dispositivo. Este modelo possui uma implementação um pouco mais complexa, vejamos:

No parâmetro "P1: JContext" iremos informar a Activity principal da nossa aplicação, para isso basta chamar o método MainActivity.

No parâmetro "P2: JHandler" temos que informar um objeto do tipo **Handle** do Android. Esse Handle tem como objetivo manipular as mensagens/threads de um outro objeto específico. Resumindo o Handle é o objeto que vai ficar "Escutando" as ações de um outro objeto, como se estivéssemos trabalhando com as mensagens do Windows para capturar um evento de um componente.

Handle Android

Para instanciarmos um Handle também chamaremos o método JavaClass.Init, e iremos precisar informar outros dois parâmetros, "looper: JLooper" e "callback: JHandler_Callback".

```
//  
function init(looper: JLooper; callback: JHandler_Callback): JHandler;
```

O parametro Looper define quem será responsável por gerenciar a fila de execuções de mensagem, e o parâmetro Callback o responsável por capturar as mensagens executadas.

Para parâmetro Looper vamos atribuir o Looper da aplicação mesmo, `TJLooper.JavaClass.getMainLooper`.

Para o parametro Callback, que é o objeto que receberá as mensagens atribuídas ao `Handle`, não temos uma classe ou método que nos instancie ou retorne o objeto pronto, temos que criar no braço, o que não é nenhum bicho de 7 cabeça... Depois que você sabe o que está fazendo.

JHandler_Callback

Nossa classe de Callback ficará da seguinte forma:

```
//  
THndCallback = class(TJavaLocal, JHandler_Callback)  
public  
    constructor Create;  
    function handleMessage(msg: JMessage): Boolean; cdecl;  
end;  
  
constructor THndCallback.Create;  
begin  
end;  
  
function THndCallback.handleMessage(msg: JMessage): Boolean;  
begin  
    Result := True;  
end;
```

Como nosso objetivo é fazer a implementação da impressora da forma mais simples possível não iremos implementar praticamente nada nesta classe.

No método `handleMessage` implementaremos apenas o `“result:= True”`, para que não tenhamos nenhum Warning, mas seria aqui que você manipularia os eventos da impressora, como um possível `OnConnect`, por exemplo. este método lembra muito o método `WndProc` da VCL.

Nesta classe também temos um **detalhe muito importante**, o Construtor, para criarmos uma classe de Callback o construtor da classe TJavaLocal não pode ser executado, então devemos declará-lo em nossa nova classe mesmo que vazio, o importante é que não devemos efetuar a chamada inherited no método.

Agora temos todos os elementos necessário para iniciar nossa variável a instância do TJBluetoothService fica assim:

```
//  
procedure TForm1.InstaciarServicoDeImpressao;  
var  
    lHandler: JHandler;  
begin  
    lHandler := TJHandler.JavaClass.init(TJLooper.JavaClass.getMainLooper,  
                                         THndCallback.Create);  
    FService := TJBluetoothService.JavaClass.init(MainActivity, lHandler);  
end;
```

Com nossa variável FService instanciada, agora é efetuar a programação das funcionalidades propriamente ditas.

Listando dispositivos pareados

Agora vamos listar os dispositivos Bluetooth pareados.

O Delphi já possui métodos que listam os dispositivos pareados, mas alguns fabricantes trabalham com classes de dispositivos própria, o que não é o caso do nosso fabricante, que retorna uma lista de objetos que implementam a interface JBluetoothDevice, que é a interface base para Devices bluetooth no Delphi.

Utilizaremos o método do fabricante e não o do Delphi, assim qualquer problema com a integração com a API da impressora será possível identificar logo na listagem dos dispositivos.

O JBluetoothDevice possui o método getPairedDev, que retorna a lista dos dispositivos, dessa forma iremos declarar uma variável global para armazenar nossa lista. O tipo de retorno do método getPairedDev é um JSet;

JSet

No tipo JSet, a manipulação da lista é um pouco mais complexa que as que estamos acostumado a utilizar no Delphi. Para manipulá-lo utilizamos o método iterator do que por sua vez retorna um tipo JIterator, e é nele que percorremos os registros.

Aí vai a receita de bolo:


```
lIt: JIterator;  
lDevicePareado: JBluetoothDevice;  
begin  
  lIt:= FDeviceSet.iterator;  
  
  while lIt.hasNext do  
  begin  
    lDevicePareado := TJBluetoothDevice.Wrap((lIt.next as ILocalObject).GetObjectID);  
    Memo1.Lines.Add(jstringtostring(lDevicePareado.getName)+'='+  
      jstringtostring(lDevicePareado.getAddress));  
  
    if jstringtostring(lDevicePareado.getName) = 'HHW-UART-S10' then  
    begin  
      FDevice := lDevicePareado;  
      Memo1.Lines.Add('Pareamento de impressora Localizado');  
    end;  
  end;  
end;
```

Cada registro no nosso iterator retorna um dispositivo pareado, que é atribuímos para a variável local `lDevicePareado`.

Para visualizarmos os Devices pareados buscamos o Nome (`lDevicePareado.getName`) e o endereço de Mac (`lDevicePareado.getAddress`), e jogamos para o Memo na tela. Por fim identifico o nome do meu dispositivo referente a impressora e atribuo a uma variável global chamada `FDevice`.

Estou identificando qual Device pareado é a impressora apenas efetuando um teste pelo nome do Pareamento, em sua aplicação você deve montar uma tela ou combo para que o usuário identifique qual dos itens da lista é a impressora e o selecione.

Outro ponto importante que vale mencionar é que o método `getPairedDev` não vai localizar a sua impressora, ele apenas lista os dispositivos pareados, dessa forma o Android listará todos os dispositivos pareados, independente de estarem ligados ou não. Se listarmos os dispositivos mesmo com a impressora desligada ela poderá aparecer em nossa lista por já ter sido pareada anteriormente.

Observe também que os retornos se dão em **JString**, ou seja Strings do Java, dessa forma devemos usar o método `jstringtostring` da unit `Androidapi.Helpers` para convertê-las em strings do Delphi.

Conexão com a Impressora

Tendo o Device mapeado vamos fazer a conexão, para isso basta chamar o método `connect` da variável `FService` passando como parâmetro o Device que queremos conectar.

```
//  
FService.connect(FDevice);
```

Ponto importante, as conexões com os dispositivos bluetooth demoram, em meus testes uma média de 2 segundo, o que representa quase um eternidade em termos de processamento, e o suficiente para você não poder efetuar um teste de impressão imediatamente após a conexão.

Status da conexão

Neste exemplo eu coloquei um botão para retornar o status da impressora, poderia testar através de um Timer ou mesmo na classe de Callback, mas por simplicidade coloquei em um botão.

```
//  
function TForm1.ObterStatus: string;  
var  
    lStateCod: integer;  
begin  
    lStateCod := FService.getState;  
  
    if lStateCod = TJBluetoothService.JavaClass.STATE_CONNECTED then  
        Result := 'CONNECTED'  
    else if lStateCod = TJBluetoothService.JavaClass.STATE_CONNECTING then  
        Result := 'CONNECTING'  
    else if lStateCod = TJBluetoothService.JavaClass.STATE_LISTEN then  
        Result := 'LISTEN'  
    else if lStateCod = TJBluetoothService.JavaClass.STATE_NONE then  
        Result := 'NONE'  
    else Result := 'Não identificado';  
end;  
  
procedure TForm1.btnStatusClick(Sender: TObject);  
begin  
    try  
        Memo1.Lines.Add('Status: '+ ObterStatus);  
    except on E:Exception do  
        Memo1.Lines.Add(e.Message);  
    end;  
end;
```

Com este teste posso verificar a qualquer momento estado da impressora.

Se ela estiver com o Status `STATE_CONNECTED` podemos enviar o comando de impressão.

Impressão

```
//  
procedure TForm1.Imprimir(pTexto: string);  
begin  
    FService.sendMessage(StringToJString(pTexto),StringToJString('GBK'));  
end;  
  
procedure TForm1.btnImprimirClick(Sender: TObject);  
begin  
    Imprimir(edit1.Text);  
end;
```

Conclusão

A integração de qualquer dispositivo bluetooth em aplicativos mobiles feitos em Delphi na maior parte das vezes pode ser feita de maneira muito simples e rápida, mas é preciso que o programador esteja familiarizado com a API do Android.

Em nosso artigo mostramos que independente do SDK ser disponibilizado pelo fabricante apenas em Java, o Delphi possui estrutura para uma completa integração com de seus aplicativos mobiles e dispositivos bluetooth com um alto ganho de produtividade.

Reduce development time and get to market faster with RAD Studio, Delphi, or C++Builder. Design. Code. Compile. Deploy.

Start Free Trial

Free Delphi Community Edition

Free C++Builder Community Edition

Upgrade Today

5 comments 0 members are here



Paulo R4458 *over 4 years ago*

Show de bola.



Adan Araujo *over 4 years ago*

Donde puedo descargar el código fuente de ejemplo? Estoy intentando, pero tengo un AV.



Ya encuentre el problema, me inicializar el FDeviceSet, con los dispositivos conectados.



Bandeira *over 3 years ago*

Olá boa noite. Existe possibilidade de baixar o código fonte do exemplo ? Obrigado.



LuizO *over 2 years ago*

Ola, estou com dificuldade de anexar uma lib a meu projeto. voce pode dar um suporte?